AI Agent Lab — Phase + Epic + Task Roadmap

Executive Summary

This roadmap defines an execution path to evolve the current local prototype into a product-grade platform.

Approach:
- Preserve the working core (Control Plane + Worker + Redis/PostgreSQL)
- Incrementally add orchestration, dependency management, multi-agent execution, and governance layers
- End each phase with a working vertical slice, measurable acceptance criteria, and an approval gate

---

Technical Appendix

Current Progress Note (Implemented)

- Task state model was expanded in `agent-lab/control-plane/Program.cs`.
- Centralized transition validation (`TaskStatusTransitions`) was added.
- `approve/retry/status` endpoints now use centralized transition rules.
- `task_status_transitions` history table and insertion helper were added.
- Retry flow was updated to `Failed -> Retrying`.
- In `agent-lab/agent-worker/Program.cs`, transient/permanent error classification, status update retries, and an in-memory idempotency guard were implemented.
- Worker dead-letter queue baseline was added (`TASK_DEAD_LETTER_QUEUE_KEY`, `DeadLetterMessage`, enqueue on terminal failure paths).
- `tests/AgentWorker.Tests` was created, added to solution, and Worker reliability tests (12 tests) are passing.
- `tests/ControlPlane.Api.Tests` was added, with 11 status-transition tests.
- Phase 2 planning layer skeleton was added: `/plans`, `/plans/generate`, `/plans/{id}/approve` endpoints and `execution_plans` schema.
- A transaction connection-state fix was applied in transaction endpoints (`await connection.OpenAsync()`), resolving the 500 error observed during bootstrap smoke testing.
- Regression validation completed: solution build succeeds and all 23 tests pass.
- `scripts/local.ps1 bootstrap-dev -LlmProvider podman-ai-lab` was executed end-to-end; stack came up, smoke test reached `Completed`, and runtime model settings were verified against `podman-ai-lab`.

Phase 1 — Foundation Hardening

Epic 1.1: Domain and State Machine Expansion
- Task 1.1.1: Extend task status model (`Proposed, Planned, InProgress, Review, Blocked, Cancelled, Rejected`)
- Task 1.1.2: Implement centralized transition validator
- Task 1.1.3: Add `TaskTransitionHistory` table
- Task 1.1.4: Add transition metadata to API responses

Epic 1.2: Reliability and Failure Handling
- Task 1.2.1: Classify worker retries (transient/permanent)

- Task 1.2.2: Add idempotency guard (duplicate task processing)
- Task 1.2.3: Define dead-letter queue approach
- Task 1.2.4: Add compensation flow for status update failures

Epic 1.3: Testing Baseline
- Task 1.3.1: Add Control Plane API integration test project
- Task 1.3.2: Add Worker unit tests (prompt build, retry, parsing)
- Task 1.3.3: Standardize minimal local end-to-end smoke test script

Phase 1 Acceptance Criteria
- Extended state transitions return 400 for invalid paths
- Worker cannot complete the same task twice
- Critical flow tests run deterministically in local environment

---

Phase 2 — Orchestrator and Planning Layer

Epic 2.1: Request-to-Plan Transformation
- Task 2.1.1: Add `ProjectRequest` and `ExecutionPlan` data models
- Task 2.1.2: Define plan generation endpoint (`/plans/generate`)
- Task 2.1.3: Add plan approval endpoint (`/plans/{id}/approve`)

Epic 2.2: Task Dependency Engine
- Task 2.2.1: Add task dependency graph model
- Task 2.2.2: Implement topological order resolver
- Task 2.2.3: Implement blocked-task resolver rules

Epic 2.3: Approval Gates
- Task 2.3.1: Mark human-approval transitions via policy
- Task 2.3.2: Add approval log records
- Task 2.3.3: Add rejection and replanning flow

Phase 2 Acceptance Criteria
- Requests become plans, and execution does not start without approval
- Dependency-ordered tasks execute correctly
- Rejected plans are recorded in audit trail

Phase 3 — Agent Workforce

Epic 3.1: Agent Definition/Instance Split
- Task 3.1.1: AgentDefinition schema (role, permissions, model policy)
- Task 3.1.2: AgentInstance lifecycle management
- Task 3.1.3: Capability matching algorithm

Epic 3.2: Assignment and Traceability
- Task 3.2.1: Task-to-agent assignment policy
- Task 3.2.2: Agent run history records
- Task 3.2.3: Agent performance metrics (success, duration, retry)

Epic 3.3: Security Boundaries
- Task 3.3.1: Tool permission allowlist
- Task 3.3.2: Repo/path scope constraints
- Task 3.3.3: Secrets redaction verification

Phase 3 Acceptance Criteria

- At least 3 specialist roles can be assigned tasks
- Unauthorized tool calls are blocked
- Agent run history is queryable

Phase 4 — Product Interface and Governance

Epic 4.1: Chat + Dashboard
- Task 4.1.1: Conversation-to-request mapping
- Task 4.1.2: Plan/task timeline view
- Task 4.1.3: Approve/reject action screens

Epic 4.2: Audit and Operational Visibility
- Task 4.2.1: AuditEvent model and API
- Task 4.2.2: Cost tagging (project/task/model)
- Task 4.2.3: Error and incident reporting panel

Epic 4.3: Quality Gates
- Task 4.3.1: Pre-PR test gate
- Task 4.3.2: Security policy gate
- Task 4.3.3: Documentation update gate (`docs-md` -> `docs` PDF)

Phase 4 Acceptance Criteria
- Product owner can view/approve/reject plans
- Audit trail remains continuous
- Finalization cannot proceed without quality gates

Cross-Phase Dependencies

- Phase 2 is risky without Phase 1 hardening
- Phase 3 is unstable without Phase 2 planning/dependency layer
- Phase 4 governance is weak without Phase 3 agent controls

Critical dependency chain:
`State machine -> Plan graph -> Agent assignment -> Governance UI`

Two-Week Sprint Proposal (Summary)

- Sprint 1-2: Phase 1
- Sprint 3-4: Phase 2
- Sprint 5-6: Phase 3
- Sprint 7-8: Phase 4

At the end of each sprint:
1. Working demo
2. Test evidence
3. Documentation update approval question
4. If approved, synchronized update for `docs-md` and `docs` PDFs